from 0. These position numbers are used to perform operations on the list. If you want to access the list, use the print command with the list name and reference the position of the value you want to call. A few examples, along with outputs, are shown here: *https://dgit.in/ListPy.*

Values in list can also be updated or changed, like `list1[2]="society"`. The earlier value at position 2 for list1 was buildings now it has been changed to "society". Note the third line print `("list1 value 0, 3: ", list1[0], list1[3])` while calling the original list. To access two non subsequent values or not in a range the list1 function had to be used twice with 2 different positional numbers. You may think that an easier method to do would be

- `print ("list1 value 0, 3: ", list1[0,3])`

But this will give you the error `TypeError: list indices must be integers or slices, not tuple.`

Tuples are also sequences like lists. The difference being that lists can be changed after creation, tuples are immutable. The syntax between a list and tuple is also slightly different with tuples using parentheses compared to square brackets of lists.

- `tup1=("hi", "hello", 2, 4)`

To access the tuple, again use similar syntax as in list with the same positional numbering logic.

- `print ("Value at 0: ", tup[0,])`

will give the value hi. Note that even if you want call a single value, you need to add a comma after it. You can perform the same operations on a a tuple as a list. You cannot change an existing value in a tuple but you can add or append values to it by creating a new tuple. Say you want to add "how are you", "greetings" to the created `tup1`, you create `tup2` with `tup2= ("how are you", "greetings")` then create `tup3` containing both the values by appending `tup2` to `tup1` using `tup3= tup1 + tup2`. If you print the entire `tup3`, using `print(tup3)` you will see the output as `('me', 'you', 2, 'how are you', 'greetings')`.

The next data structure, **dictionary**, is a sequence of assorted values. The value is written in pair format of key : value in curly brackets. E.g.

- `dict= {'first':"digit", 'second':2018}`

The parameter first is the key and digit is the value assigned to it. You need to use the key to to call or access the value

- `print ("dict['first']")`

This will give the output digit as it matches the value to the key defined. You can also view the keys and values specified in a particular dictionary by